Team 2 (CSULB Rates)

1. **Brady Askings**
    - **Student ID**: 030813794
    - **Email**: brady.askins01@student.csulb.edu
2. **Evgin Demirci**
    - **Student ID**: 027524326
    - **Email**: Evgin.Demirci01@student.csulb.edu
3. **Kiet Nguyen**
    - **Student ID**: 027102385
    - **Email**: Kiet.Nguyen01@student.csulb.edu
4. **Veronica Nguyen**
    - **Student ID**: 028495608
    - **Email**: veronica.nguyen01@student.csulb.edu
5. **Arthur King Ordonez**
    - **Student ID**: 031662109
    - **Email**: arthurking.ordonez01@student.csulb.edu
6. **Yunjong Noh**
    - **Student ID**: 017537024
    - **Email**: yunjong.noh@student.csulb.edu

Preface

Version History

Version	Date	Changes
0.1	11/17/2024	Template Version
0.2	11/18/2024	Modified Purpose, Introduction, User Requirements & Use Cases, and System Architecture
0.3	11/19/2024	Added Glossary Terms, Use Cases, and System Architecture
1.0	11/20/2024	Initial Version

Purpose

This document serves as a comprehensive guide for the development and comprehension of the software project entitled “CSULB Rates.”

Audience

This document is intended for the following stakeholders: project managers, developers, testers, and all other individuals involved in the project lifecycle.

Introduction

Project Overview

"CSULB Rates" is a web application that allows CSULB students to rate and review certain entities on campus, including restaurants, events, labs, etc. Its primary functions encompass anonymous ratings, creating polls, and creating ratings and reviews for various places that CSULB encompasses.

Project Goals

- Keeping user ratings and reviews anonymous.
- Updating polls, ratings, and reviews accurately in real time.
- Upvote/Downvote on polls, ratings, and reviews to show post usefulness.
- Creating a visually appealing website with smooth user experience.

Glossary

- **Entity:** Anything on CSULB campus that can be rated, such as labs, events, food, cafes, concerts, etc.
- **Rating:** A rating given for any entity at school from a scale of 1-5.
- **Poll:** A question with at least two options that users can vote on.
- **Review:** A rating with a written description attached to it.
- **Upvote:** A button the user can press to express that the review or poll is useful.
- **Downvote:** A button the user can press to express that the review or poll is not useful.

User Requirements and Use Cases

User Stories

1. As a CSULB student, I want to create customizable polls with any number of options about the school's services so that I can gather opinions on a specific topic.
2. As a university student involved on campus, I want to review school events and give feedback so that the school can create events based on student's interests and opinions.
3. As a student with a CSULB ID, I want to anonymously rate and review school entities so that I can rate honestly without revealing my identity.
4. As a student, I want to search for polls and reviews by their named entities so that I can quickly find relevant information.
5. As a guest user, I want to be able to view ratings and reviews so that I can access information without having to log in.
6. As a poll or rating creator, I want to be able to delete my poll so that I can manage whether my post can be seen.
7. As a user, I want each poll or rating to be sharable so that I can easily distribute it to others.
8. As a CSULB student, I want to register so that I can share my opinions regarding the places of the university.
9. As a critic, I want to modify my previous ratings so that I don't have to make another review.
10. As a student, I want to be able to vote on a post's usefulness so that I can report and see content that is more relevant to me.

Use Case: Creating a Poll

Identifier	UC-1: Create an opinion poll question
Purpose	Create a poll asking for opinions and preferences of students for facilities and school events
Requirements	Users should be able to create a poll question
Development Risks	None
Pre-conditions	User is logged in User is on the "Create a Poll" page
Post-conditions	User's poll is created and displayed

Table 1: Typical Course of Action

Seq#	Actor's Action	System's Response
1	User writes a poll question with multiple selectable answers if applicable	
2	User presses submit	
3		Check the validity of inputs
4		Saves in database and reloads page to show the question was successfully posted
5		Return control to user

Table 2: Alternate Course of Action

Seq#	Actor's Action	System's Response
1	User selects a common template for creating poll questions instead of writing it manually	
2		Display the poll question template
3	User fills out the template and presses submit	
4		Check the validity of inputs
5		Saves in database and reloads page to show the question was successfully posted
6		Return control to user

Table 3: Exceptional Course of Action

Seq#	Actor's Action	System's Response
1	User writes an incomplete poll question	
2	User presses submit	
3		Check the validity of inputs
4		Display error message saying that the poll question cannot be incomplete
5		Redirects user back to the "Create a Poll" page

System Architecture

Components

1. **Frontend**: Web-based user interface (built with React).
2. **Backend**: RESTful API (built with Node.js and Express).
3. **Database**: MongoDB for NoSQL data storage.
4. **Authentication**: JSON Web Tokens (JWT)

Architectural Pattern

We used the Model-View-Controller pattern for our system since we built this project using the MERN (MongoDB, Express, React, Node.js) stack.

Model = MongoDB, handles data storage

View = React, responsible for front-end showcasing

Controller = Express.js, handles routing and API requests

Below is a diagram that shows how we applied it to our system.